

ota_landscape_report

Helix OTA — Landscape Report & Engine-Selection Synthesis

Purpose: Synthesize 8 research notes into a single decision-grade landscape report for Helix OTA. **Locked strategy:** *native Android A/B (AOSP update_engine) + custom Go control plane; wrap OSS only where it adds value.* **Mandated stack:** Go + Gin + Brotli + HTTP/3→HTTP/2, REST-primary; PostgreSQL + MinIO/S3. **Phase 1 target:** Android 15 first (Orange Pi 5 Max / RK3588 class). Linux/universal is a later phase.

Author: Lead architect (synthesis) · **Date:** 2026-06-07 · **Revision:** 2 **Fixed (rev 2):** Softened header anti-bluff claim; flagged that role totals are NOT cross-comparable; scored SWUpdate into the Linux-phase matrix (removed its exclusion note); qualified Uptane confidence as high (spec) / low (impl); added DMF/AMQP-deviation-needs-ADR note; cross-referenced the new Android deep-dive notes. **Anti-bluff rule:** every score traces to a source note (cited by slug). Items the notes flagged as unverified are carried forward as **UNVERIFIED**. This report makes no new external/empirical claims; design recommendations are derived from the notes.

Table of Contents

1. Metadata Table
 2. Scored Comparison Matrix
 - 2.1 Scoring rubric
 - 2.2 Matrix
 - 2.3 Per-criterion justifications (traceable)
 3. Recommendations
 - 3.1 Device-side Android
 - 3.2 Server control-plane: wrap or build
 - 3.3 Trust framework
 4. What We Will NOT Use (and Why)
 5. Open / UNVERIFIED Items to Close Before ADR-0001
 6. Source Notes
-

1. Metadata Table

Stack	Slug	Category	License	Maturity (per note)	Android-15 fit	Con
AOSP update_engine (native A/B + Virtual A/B)	aosp-update-engine	On-device updater	Apache-2.0	Very mature; A/B since Android 7/8, Virtual A/B GMS req 11+	Excellent (this <i>is</i> the path)	high
Eclipse hawkBit	eclipse-hawkbit	Rollout/campaign back end	EPL-2.0	Mature/active; 1.0.3 (2025-04-09), API-stable	Good at orchestration seam (OS-agnostic)	med
TUF + go-tuf/v2	tuf-go-tuf	Trust/metadata framework	Apache-2.0	Mature/active; v2.4.2 (2026-05-19); CNCF spec	Server strong / device WEAK (no verified Android client)	med
Uptane	uptane	Trust/metadata framework (TUF-extended)	Open (IEEE-ISTO 6100.1.0.0); aktualizr MPL-2.0	Spec high; impl uneven (aktualizr last tag Feb 2020)	Weak as product, useful as model	high (spec (imp
Mender	mender	Client+server OTA (embedded Linux)	Open-core (client Apache-2.0; server Enterprise proprietary)	High; widely deployed	Effectively zero (no Android)	high
RAUC	rauc	On-device A/B updater (embedded Linux)	LGPL-2.1-or-later	Mature; Steam Deck; v1.15.2 (2026-03-27)	None (Linux only)	med
SWUpdate	swupdate	On-device update engine (embedded Linux)	GPL-2.0-only	Mature; broad Yocto/OE adoption; 2026.05 (2026-05-29)	None (Linux only)	med
OSTree / libostree	ostree	File-based atomic deploy (Linux)	LGPL-2.1-or-later	Mature; Silverblue/ CoreOS/ RHEL Edge	Poor for AOSP path	med
Commercial/OSS fleet (balena, Torizon,	commercial-oss-fleet	Fleet platforms	Mixed (mostly open-client /	All production; backends proprietary SaaS	Only Memfault hits AOSP A/B	high

Stack	Slug	Category	License	Maturity (per note)	Android-15 fit	Con
Memfault, Foundries.io)			closed- backend)			

2. Scored Comparison Matrix

2.1 Scoring rubric

Scores are **1–5**, judged **relative to Helix’s locked strategy and mandated stack** (not in the abstract). Higher = better fit for Helix.

- **5** = directly satisfies / near 1:1 match to Helix’s locked design.
- **4** = strong fit, minor adaptation.
- **3** = usable with meaningful integration work, or a partial fit.
- **2** = weak fit; significant mismatch or only pattern-level value.
- **1** = does not apply / disqualifying for the relevant phase.

Criteria: **mechanism, rollback, staged-rollout, A/B, Android-15 fit, Go fit, wrap-ability, license, maturity.**

2.2 Matrix

Stack (slug)	Mechanism	Rollback	Staged-rollout	A/B	Android-15 fit	Go fit	Wrap-ability	License	Ma
aosp-update-engine	5	5	1	5	5	3	5	5	
eclipse-hawkbite	5	2	5	1	4	5	5	5	
tuf-go-tuf	4	3	2	1	2	5	4	5	
uptane	4	4	4	1	2	3	2	4	
commercial-oss-fleet	3	4	5	3	3	2	2	2	
rauc	4	4	1	5	1	3	4	4	
swupdate	4	3	3	3	1	3	4	4	
ostree	4	4	1	1	1	2	4	4	
mender	4	4	2	3	1	2	1	2	

Totals are NOT comparable across roles and must not be quoted standalone. Each stack is scored against Helix’s locked strategy *for its own role* (device-apply vs. control-plane vs. trust), so a /45 total only ranks candidates *within the same role* — comparing an on-device updater’s total to a control-plane’s total is apples-to-oranges. The total is a coarse within-role ranking aid only. The decision is **role-specific** (device vs. control-plane vs. trust), so read §2.3 and §3 — not just the totals. A low A/B or Android

score is *expected and not disqualifying* for a control-plane or trust-layer candidate (e.g., hawkBit, TUF), because those roles don't perform the on-device A/B apply.

2.3 Per-criterion justifications (traceable)

Each cell below = one-line justification + citing note. “—” criteria that are N/A for a role still get a literal score per the rubric.

aosp-update-engine

- **Mechanism (5):** On-device daemon writes inactive slot via boot_control HAL; Virtual A/B COW snapshots; payload.bin from ota_from_target_files. [aosp-update-engine]
- **Rollback (5):** Automatic bootloader revert on failed boot/dm-verity (update_verifier before zygote). [aosp-update-engine]
- **Staged-rollout (1):** None — AOSP applies whatever payload it's handed; all rollout logic is the integrator's. [aosp-update-engine]
- **A/B (5):** Native A/B and Virtual A/B both driven by applyPayload. [aosp-update-engine]
- **Android-15 fit (5):** This *is* the native Android update path; thin wrapper over applyPayload. [aosp-update-engine]
- **Go fit (3):** N/A on device (Java/AIDL + C++); Go fits the server side that builds/serves payload.bin + owns rollout/telemetry. [aosp-update-engine]
- **Wrap-ability (5):** Stable supported client API + AOSP SystemUpdaterSample reference. [aosp-update-engine]
- **License (5):** Apache-2.0. [aosp-update-engine]
- **Maturity (5):** Standard since Android 7/8; production-wide. [aosp-update-engine]

eclipse-hawkbit

- **Mechanism (5):** Three clean APIs (Management/DDI/DMF); threshold-gated cascading deployment groups deploying Distribution Sets. [eclipse-hawkbit]
- **Rollback (2):** No firmware/app rollback of its own beyond halting a rollout (emergency shutdown); per-device boot rollback is AOSP's job. [eclipse-hawkbit]
- **Staged-rollout (5):** Cascading groups with success/trigger + error thresholds, approval gate, start/pause/resume — near 1:1 to locked Helix design. [eclipse-hawkbit]
- **A/B (1):** No native A/B; artifacts opaque. [eclipse-hawkbit]
- **Android-15 fit (4):** OS-agnostic; DDI is plain HTTPS poll + token + feedback, trivially implementable on Android; client bridges DDI→applyPayload. [eclipse-hawkbit]
- **Go fit (5):** Explicitly designed to be driven by a 3rd-party app over the Management API — the Go control plane becomes that app. [eclipse-hawkbit]
- **Wrap-ability (5):** Clean seam; hard orchestration reused, custom code stays in Go; sits behind the Go control plane, never public. [eclipse-hawkbit]
- **License (5):** EPL-2.0, business-friendly. [eclipse-hawkbit]
- **Maturity (4):** Eclipse Mature; 1.0.x API-stable; 1.0.3 (2025-04-09). [eclipse-hawkbit]

tuf-go-tuf

- **Mechanism (4):** Signed metadata (root/timestamp/snapshot/targets) verified in fixed order; threshold + offline keys. [tuf-go-tuf]
- **Rollback (3):** Prevents rollback *attacks* (version monotonicity); does not execute device rollback. [tuf-go-tuf]
- **Staged-rollout (2):** No native primitive; delegations can partition target namespace, but phasing stays in control plane. [tuf-go-tuf]
- **A/B (1):** N/A — does not perform slot switching. [tuf-go-tuf]
- **Android-15 fit (2):** No verified production Android/Kotlin TUF client; on-device path (gomobile/JNI vs hand-rolled Kotlin) unresolved — dominant adoption risk. **UNVERIFIED** client. [tuf-go-tuf]
- **Go fit (5):** Idiomatic go-tuf/v2, Apache-2.0, clean generics, single dep tree; server publish path is a clean wrap. [tuf-go-tuf]
- **Wrap-ability (4):** High on server, additive/non-invasive (artifacts stay byte-identical); device-side blocked by missing client. [tuf-go-tuf]
- **License (5):** Apache-2.0 (verified 2026-06-07). [tuf-go-tuf]
- **Maturity (4):** Active; v2.4.2 (2026-05-19); CNCF-graduated spec; adopter list **UNVERIFIED**. [tuf-go-tuf]

uptane

- **Mechanism (4):** Dual-repo (offline Image + online Director) + four TUF roles; per-device metadata. [uptane]
- **Rollback (4):** Anti-rollback/anti-freeze core guarantees (metadata layer; distinct from device A/B). [uptane]
- **Staged-rollout (4):** Director mints per-device metadata from inventory DB — strong conceptual fit for percentage staging (orchestration still Helix's). [uptane]
- **A/B (1):** Orthogonal; provides no A/B itself. [uptane]
- **Android-15 fit (2):** No first-class AOSP client; bespoke integration; **UNVERIFIED** any production Android+Uptane ships. [uptane]
- **Go fit (3):** No Go reference impl (aktualizr is C++), but the metadata layer is straightforward to implement in Go. [uptane]
- **Wrap-ability (2):** Low as whole-system wrap (aktualizr C++/Yocto/OSTree-coupled); high only as a borrowable design. [uptane]
- **License (4):** Open standard; aktualizr MPL-2.0. [uptane]
- **Maturity (3):** Spec maturity high; implementation cadence low (aktualizr last tag Feb 2020). [uptane]

commercial-oss-fleet (balena / Torizon / Memfault / Foundries.io)

- **Mechanism (3):** Four distinct models; only Memfault drives AOSP UpdateEngine; others are Linux-filesystem/container. [commercial-oss-fleet]
- **Rollback (4):** Foundries.io/Torizon auto bootloader rollback after 3 failed boots; Memfault uses AOSP A/B slot rollback. [commercial-oss-fleet]
- **Staged-rollout (5):** Foundries.io waves+canary+cancel (group/UUID/percentage); Memfault cohorts + one-click abort — strong patterns to copy. [commercial-oss-fleet]
- **A/B (3):** Memfault = native Android UpdateEngine A/B; others not AOSP. [commercial-oss-fleet]
- **Android-15 fit (3):** Only Memfault targets AOSP A/B (Bort SDK + poll-releases-HTTP-API); others' delivery layer not reusable. [commercial-oss-fleet]

- **Go fit (2):** None ship a reusable Go control plane; only orchestration patterns transfer. [commercial-oss-fleet]
- **Wrap-ability (2):** Low as whole platforms (proprietary SaaS / feature-gated OSS backends); portable assets are clients + patterns. [commercial-oss-fleet]
- **License (2):** Control planes are the lock-in: Memfault/Foundries backends proprietary SaaS; openBalena AGPLv3 single-user beta; Torizon CE non-prod. [commercial-oss-fleet]
- **Maturity (4):** All four production-grade in commercial use. [commercial-oss-fleet]

rauc

- **Mechanism (4):** C daemon installs signed SquashFS bundles into redundant slots; HTTP(S) streaming via NBD+dm-verity; D-Bus control. [rauc]
- **Rollback (4):** Bootloader-driven fallback on repeated boot failure; mark-good/mark-bad confirmation. [rauc]
- **Staged-rollout (1):** None — RAUC ships no server/orchestration (a fit for custom control plane, not a feature). [rauc]
- **A/B (5):** Native A/B (and multi-slot) is core design. [rauc]
- **Android-15 fit (1):** Embedded Linux only; no Android/update_engine/Virtual A/B. [rauc]
- **Go fit (3):** No server to displace; delivery is plain HTTP+Range (Gin-able), but device control is D-Bus, not Go (only unmaintained constants-only Go helper). [rauc]
- **Wrap-ability (4):** Clean device-install/orchestration separation; async D-Bus maps to an OS-adapter; cost = self-maintained D-Bus binding + bootloader config. [rauc]
- **License (4):** LGPL-2.1-or-later (dynamic-linking caveat; D-Bus/CLI usage sidesteps it). [rauc]
- **Maturity (4):** Mature; Steam Deck/SteamOS; v1.15.2 (2026-03-27). [rauc]

swupdate

- **Mechanism (4):** Handler-driven streaming install pipeline; cpio .swu + sw-description (libconfig or Lua parser); flexible but not Helix's native Android mechanism. [swupdate]
- **Rollback (3):** Delegated to the bootloader env (U-Boot/GRUB/EFI Boot Guard) and assembled by the integrator; no built-in slot-confirm state machine like RAUC/update_engine. [swupdate]
- **Staged-rollout (3):** No native rollout orchestration in the engine, but suricata + hawkBit DDI/wfx gives a real path when paired with a server (above RAUC's 1, below hawkBit's 5 since the engine itself does none). [swupdate]
- **A/B (3):** Double-copy via Software collections + bootloader env, but convention/config rather than a first-class slot manager (below RAUC's 5). [swupdate]
- **Android-15 fit (1):** No Android target at all; embedded-Linux only; irrelevant to the MVP. [swupdate]
- **Go fit (3):** No Go reuse of the engine, but the server contract is a trivial Go target (general-purpose HTTP) or a documented one (hawkBit DDI). [swupdate]
- **Wrap-ability (4):** Excellent as a wrapped Linux apply engine (one-shot or suricata; Lua-extensible; integrator owns the server); one point below RAUC because A/B/rollback orchestration is less turnkey. Zero for Android. [swupdate]

- **License (4):** GPL-2.0-only copyleft; fine for a separate on-device daemon driven over a protocol, but distribution of modified binaries carries obligations (RAUC's LGPL is slightly more permissive). [swupdate]
- **Maturity (4):** Long-lived, actively maintained (2026.05, 2026-05-29); broad Yocto/OE adoption; hawkBit/wfx/delta features maintained. [swupdate]

ostree

- **Mechanism (4):** Content-addressed object store; parallel hardlinked deployments + atomic bootloader pointer swap; static deltas. [ostree]
- **Rollback (4):** Previous deployment always retained; pinning exempts known-good from GC (exact `ostree admin rollback` command **UNVERIFIED**). [ostree]
- **Staged-rollout (1):** No built-in engine; pull-based; rollout is entirely Helix control plane's. [ostree]
- **A/B (1):** Not A/B — parallel deployments on one partition; does not replace Android native A/B. [ostree]
- **Android-15 fit (1):** Linux-userspace; aboot backend is Linux-on-aboot, not AOSP. [ostree]
- **Go fit (2):** No first-class Go binding; CLI orchestration or cgo/GI. [ostree]
- **Wrap-ability (4):** Server is `ostree` CLI → archive repo over plain HTTP, drops into Go + S3/MinIO; client is the `ostree` binary behind a Linux adapter. [ostree]
- **License (4):** LGPL-2.1-or-later. [ostree]
- **Maturity (4):** Mature, production-proven (Silverblue/CoreOS/RHEL Edge); ecosystem shifting to bootc. [ostree]

mender

- **Mechanism (4):** Go client polls HTTPS; dual A/B rootfs (bootloader-driven) + Update Modules; Go microservices server (Mongo/NATS/Traefik). [mender]
 - **Rollback (4):** Automatic device-level bootloader fallback; power-loss safe. [mender]
 - **Staged-rollout (2):** “Phased rollouts” + dynamic grouping exist but are Enterprise-tier; nothing in the free OSS plan. [mender]
 - **A/B (3):** Yes for embedded Linux via bootloader-driven dual A/B; not Android update engine. [mender]
 - **Android-15 fit (1):** Android absent from supported platforms — disqualifying. [mender]
 - **Go fit (2):** Client Go, but server is Mongo/NATS/Traefik microservices — conflicts with Gin+Postgres+MinIO+REST mandate. [mender]
 - **Wrap-ability (1):** Cannot wrap an engine that doesn't run on Android; server conflicts with stack + open-core gating. [mender]
 - **License (2):** Open-core; key orchestration features Professional/Enterprise-only (proprietary). [mender]
 - **Maturity (5):** High; mature, widely deployed in embedded Linux/IoT. [mender]
-

3. Recommendations

Decision summary aligned to the locked strategy: **build the Go control plane, wrap AOSP on-device, wrap hawkBit for orchestration if its seam confirms, and adopt TUF (go-tuf/v2) server-side now with a device-client spike gating mandatory enforcement.**

3.1 Device-side Android

ADOPT AOSP update_engine (native A/B + Virtual A/B); wrap, do not replace. [aosp-update-engine]

Device-side deep-dive references: the implementation details behind this recommendation are expanded in the Android notes: [android15-virtual-ab.md](#) (Virtual A/B / COW snapshots on Android 15), [android-update-engine-api.md](#) (UpdateEngine.applyPayload surface and callbacks), and [android-avb-rollback.md](#) (AVB and rollback protection). Consult these when building the thin client and confirming RK3588 / Orange Pi 5 Max board specifics.

- Build a **thin Android client** around UpdateEngine.applyPayload that receives {url, offset, size, FILE_HASH, FILE_SIZE, METADATA_HASH, METADATA_SIZE} from the Go control plane and relays onStatusUpdate / onPayloadApplicationComplete as telemetry. Do **not** re-implement the apply/verify/rollback path — AOSP already provides atomic inactive-slot writes and automatic bootloader rollback on failed boot/dm-verity. [aosp-update-engine]
- Helix's device-side investment goes into: the **build pipeline** (ota_from_target_files + AVB/release signing key custody), **streaming-correct serving** (ZIP_STORED packaging + HTTP Range), and **reconciling daemon state across process death**. [aosp-update-engine]
- **Pattern to borrow:** Memfault's AOSP path (poll-a-releases-HTTP-API → drive UpdateEngine A/B → report install/boot result) is the closest existing blueprint for the device-client shape. Add a **post-boot health-confirmation window** (beyond AOSP's boot-success mark) before marking a release "good," with automatic canary abort on crash/health regression. [commercial-oss-fleet]
- **Board caveat:** some exact Android-15 constants/AIDL headers for RK3588 / Orange Pi 5 Max remain **UNVERIFIED** and need board-level confirmation. [aosp-update-engine]

3.2 Server control-plane: wrap or build

BUILD the Helix control plane in Go (Gin, REST-primary, Brotli, HTTP/3→HTTP/2, PostgreSQL, MinIO/S3) — and WRAP Eclipse hawkBit for the staged-rollout/campaign engine (front-runner, gated). [eclipse-hawkbit]

Rationale: - The locked strategy is *build the control plane, wrap OSS only where it adds value*. The single piece of hard, battle-tested orchestration worth wrapping is **threshold-gated cascading rollouts with emergency shutdown**, which hawkBit provides near 1:1 to the Helix staged-rollout design. The Go control plane becomes the "3rd-party app" hawkBit is explicitly designed to be driven by (Management API), DDI faces the Android client, DMF stays optional. [eclipse-hawkbit] -

Leanest MVP topology: hawkbit-monolith + PostgreSQL + DDI only (no RabbitMQ unless DMF push is needed) + MinIO/S3 for artifacts. hawkBit sits **behind** the Go control plane, never exposed publicly. [eclipse-hawkbit] - **Cost / trade-off:** operating a JVM/Spring-Boot runtime alongside Go is the main downside. The decision must be gated on confirming whether the Go consumer can **subscribe to management events or must poll**, and the exact rollout create/start/pause/resume API surface (both **UNVERIFIED**). [eclipse-hawkbit] - **If hawkBit is rejected at ADR-0001** (e.g., team declines the JVM operational cost), **build the rollout engine natively in Go** and copy Foundries.io's wave state machine (init→canary→expanding→complete/cancelled) + device-tag cohorts (group/UUID/percentage targeting) and Memfault's server-side one-click abort. These patterns are reusable even though no platform ships a reusable Go control plane. [commercial-oss-fleet] - **Do not wrap a fleet platform backend or Mender's server** — see §4.

3.3 Trust framework

ADOPT TUF via go-tuf/v2 as the server-side metadata/trust layer now; gate mandatory device-side enforcement behind an Android-client spike. Borrow Uptane's dual-repo model as a stretch hardening goal. [tuf-go-tuf] [uptane]

- go-tuf/v2 is idiomatic Go, Apache-2.0, actively maintained (v2.4.2, 2026-05-19), and **additive/non-invasive**: artifacts (payload.bin / .zip) stay byte-identical and keep their AOSP/payload signature; TUF treats them as opaque target files. It closes attack classes plain per-artifact signing leaves open (rollback, indefinite freeze, mix-and-match, malicious-mirror denial, key-compromise resilience via thresholds + offline keys) and serves over the existing HTTP/3→HTTP/2 + Brotli path. [tuf-go-tuf]
- **The blocker is device-side, not the library**: no verified production-grade Android/Kotlin TUF client exists (**UNVERIFIED**); the on-device path (gomobile/JNI vs hand-rolled Kotlin) is the dominant adoption cost. [tuf-go-tuf]
- **Sequencing**: (1) prototype server publish + a Go reference refresh client end-to-end; (2) ADR + spike for the on-device client; (3) define offline key-custody ceremony (HSM/KMS, threshold, rotation); (4) only then make device-side TUF verification mandatory. Keep SHA-256 + per-artifact signature + the A/B engine's payload check as defense-in-depth regardless. [tuf-go-tuf]
- **Stretch goal — Uptane model, not Uptane the product**: implement a TUF/Uptane-inspired **Director** + **Image** split (online Director from inventory DB + offline-keyed Image repo) in Go to get compromise resilience where security degrades only if *both* repos are compromised — directly hardening the online Go control plane. Skip the Primary/Secondary ECU tier (vacuous for independent full-power Android devices). Do **not** adopt aktualizr. [uptane]

4. What We Will NOT Use (and Why)

Stack	Decision	Why (traceable)
Mender (client + server)	Reject for adoption/wrapping; keep as design reference only	No Android support whatsoever; A/B is bootloader-driven, not AOSP update_engine; key orchestration (phased

Stack	Decision	Why (traceable)
RAUC	Not for Android Phase 1; shortlist for the later Linux/universal phase	rollouts, dynamic grouping, RBAC, delta gen) is Professional/Enterprise-only open-core; server stack (Mongo/NATS/Traefik) conflicts with the locked Gin+Postgres+MinIO+REST mandate. Cannot wrap an engine that doesn't run on Android. Retain only the Update Modules pattern, dual-A/B commit/rollback semantics, polling-security model, and REST ergonomics. [mender] Embedded-Linux only; zero Android/update_engine/ Virtual A/B value now. Strongest standards-aligned server-agnostic A/B engine for Linux later (CMS/X.509/ HSM > ad-hoc SHA-256+sig; built-in HTTP-Range streaming fits the Go control plane). Carry into ADR-0001 gated by a hands-on spike + re-verification of flagged facts. [rauc]
OSTree / libostree	Not for the Android device path; shortlist for the Linux phase only	Linux-userspace by design; about backend is Linux-on-about, not AOSP; not A/B and no built-in staged rollout. Strong Linux OS-adaptor candidate (pinning + static deltas; archive-over-HTTP drops into Go+S3/MinIO). Evaluate head-to-head against bootc given the ecosystem shift. [ostree]
Uptane / aktualizr (as a product)	Reject aktualizr; adopt the security model selectively	aktualizr is C++/Yocto/OSTree-coupled (mismatched with Go+Android), low release cadence (last tag Feb 2020), no Go reference impl, no first-class Android integration (UNVERIFIED any production Android+Uptane ships). Borrow the dual-repo +

Stack	Decision	Why (traceable)
Commercial/OSS fleet platforms (balena, Torizon, Memfault, Foundries.io)	Reject wholesale adoption; synthesize patterns + reuse open clients only	offline/online key split in Go instead. [uptane] Control planes are the lock-in: Memfault and Foundries.io backends are proprietary SaaS with no self-host; openBalena is AGPLv3 single-user beta; Torizon OTA Community Edition is no-auth/non-prod. Helix's open Go control plane is a genuine differentiator vs the dominant open-client/closed-backend model. Reuse only: Memfault's AOSP device-client shape, Foundries.io's wave/canary state machine, TUF-style signing baseline, balena's update-lock veto. [commercial-oss-fleet] DMF push requires a RabbitMQ broker; DDI-only avoids the AMQP broker. Use DDI-only unless management push proves necessary. Note: enabling DMF (AMQP/RabbitMQ) would be a deviation from the locked REST-primary mandate and must not be turned on implicitly — it requires its own ADR justifying the added AMQP broker and transport. [eclipse-hawkebit] hawkBit has no TUF understanding and no app/firmware rollback orchestration of its own beyond halting a rollout; trust is TUF/go-tuf's job and per-device boot rollback is AOSP's. [eclipse-hawkebit]
hawkBit DMF / RabbitMQ (within an adopted hawkBit)	Defer for MVP	
hawkBit as the trust layer	Do not rely on it for TUF-grade trust	

swupdate is now scored in §2.2/§2.3 (Linux-phase, total 29/45) alongside RAUC and OSTree. Like RAUC, it is **not for Android Phase 1** (no Android target) but is a **shortlist Linux-phase device-apply engine** — on par with RAUC, ahead of OSTree for this role, with A/B/rollback being integrator-assembled (Software collections + bootloader env). Carry into ADR-0001 gated by a hands-on spike. [swupdate]

5. Open / UNVERIFIED Items to Close Before ADR-0001

1. **hawkBit management-side event push vs. polling** for the Go consumer, and the exact rollout **create/start/pause/resume** API surface (some rollout JSON fields / polling defaults / endpoint verbs flagged UNVERIFIED — docs are a client-rendered SPA). [eclipse-hawkbit]
 2. **Android TUF client** — no verified production Kotlin/JVM client; resolve gomobile/JNI vs hand-rolled Kotlin via a spike; this dominates TUF adoption cost. [tuf-go-tuf]
 3. **TUF client-flow API** (metadata/updater, metadata/trustedmetadata) signatures not fully verified (pkg.go.dev truncated); TUF adopter list self-reported. [tuf-go-tuf]
 4. **Android-15 / RK3588 board specifics** — exact update_engine constants/AIDL headers and Virtual A/B figures (~45% full / ~55% incremental savings) UNVERIFIED; confirm at board level. [aosp-update-engine]
 5. **Uptane** — no Go/Rust reference impl confirmed; Standard 2.1.0 release date and AGL integration status UNVERIFIED. [uptane]
 6. **RAUC** — interaction of NBD/HTTP-Range streaming with Helix Brotli/HTTP3; several quantitative claims (binary size, benchmark, crypt AES mode, latest version) need confirmation before an ADR locks. [rauc]
 7. **OSTree** — GC retention defaults, delta size ratios, storage overhead, --json status, exact rollback command, and bootc head-to-head all UNVERIFIED. [ostree]
 8. **swupdate** — now scored (Linux-phase, 29/45); before ADR-0001 confirm exact streaming-capable handler list, crypto provider identifiers / encryption cipher-mode, whether wfx is built by default, and HTTP/3-via-libcurl availability (all flagged UNVERIFIED in the note). [swupdate]
-

6. Source Notes

All under docs/research/main_specs/research/stacks/:

- [aosp-update-engine.md](#) — AOSP update_engine / native A/B + Virtual A/B (confidence: high)
- [eclipse-hawkbit.md](#) — Eclipse hawkBit (confidence: medium)
- [tuf-go-tuf.md](#) — TUF + go-tuf/v2 (confidence: medium)
- [uptane.md](#) — Uptane (confidence: high)
- [mender.md](#) — Mender (confidence: high)
- [rauc.md](#) — RAUC (confidence: medium)
- [ostree.md](#) — OSTree / libostree / rpm-ostree (confidence: medium)
- [commercial-oss-fleet.md](#) — balena / Torizon / Memfault / Foundries.io (confidence: high)
- [swupdate.md](#) — SWUpdate (embedded-Linux engine; Linux-phase only) (confidence: high on architecture / medium on edition-specific facts)

Android device-side deep-dive notes (supporting §3.1, AOSP/device-side):

- [android15-virtual-ab.md](#) — Android 15 Virtual A/B / COW snapshots
- [android-update-engine-api.md](#) — UpdateEngine.applyPayload API surface & callbacks

- [android-avb-rollback.md](#) — Android Verified Boot (AVB) & rollback protection